

TEACHING HANDBOOK · 教师版

MiniMind 大语言模型 全流程教学讲义

从 Tokenizer、模型与数据契约，到 Pretrain、对齐、Agentic RL 与部署

适配 24 学时核心课与 40 学时完整版

教学主张

把 64M 小模型当作一台“LLM 显微镜”：每节课都能看见输入契约、
目标函数、训练产物与验收证据。

源文件：Hong-gblbcy/minimind · MiniMind-完整学习实践教程.md

分析版本：fork commit 2e2ed4cc（2026-07-28）；源文基于 upstream 89d674b8（2026-07-23）

编制日期：2026-08-02

使用说明

这不是原教程的摘要，而是一份可直接进入课堂的教师版课程设计。

原文件覆盖了 MiniMind 主线几乎所有公开模块，但内容同时承担源码说明、操作手册和排障索引。教学稿将其重组为“概念依赖—课堂活动—实验验收—评价证据”的闭环，帮助教师决定先讲什么、讲到什么深度，以及如何判断学生是否真正学会。

适用对象与先修要求

- 适用对象：具备 Python 基础、希望系统理解 LLM 训练全链路的高年级本科生、研究生、工程师或教师培训班。
- 最低先修：能阅读 PyTorch 张量代码；理解矩阵乘法、softmax、交叉熵和基本梯度下降。
- 建议先修：熟悉 Linux 命令行、虚拟环境、JSONL 和 GPU/显存的基本概念。
- 课程边界：MiniMind 是教学型小模型，不是通用大模型替代品；所有性能与安全结论都必须由本地评测支持。

统一授课框架 每个实验都用四问收口：输入契约是什么？核心算法改了什么？产物保存在哪里？用什么证据验收？

源文件知识点分析

源文件共 30 个主节、约 3,600 行，知识密度高。若按原文顺序直接授课，学生容易把命令记忆当成概念理解。下面先把知识点压缩为八个教学域，再明确先后依赖。

教学域	核心知识点	课堂难点	可观察证据
协议基础	ByteLevel BPE、token ID、Chat Template、thinking/tool 标签	special token 与 added token；三类长度边界	能解释模板到 token 流
模型结构	Decoder-only、RMSNorm、GQA、RoPE、KV Cache、SwiGLU、MoE	张量形状；198M-A64M；aux loss	能画 forward 并标注形状
数据契约	Pretrain/SFT/DPO/RLAIF/Agent 五类 Dataset、mask、截断	assistant-only labels；全 -100 风险	能解码 active labels
训练工程	AdamW、余弦学习率、混合精度、梯度累积、DDP、checkpoint/resume	全局 batch；近似恢复；多卡边界	能解释日志与产物
基础训练	Causal LM、Pretrain、Full SFT、LoRA、白盒蒸馏	模型内 shift；分支不是线性流水线	能完成 smoke 与固定回归
偏好与 RL	DPO、reward、rollout、PPO、GRPO、CISPO	序列分数、KL、组内方差、reward hacking	能读 loss 与诊断退化
Agent 与系统	多轮工具轨迹、Tool Calling、API、WebUI、转换、部署、量化	schema 一致性；mask；流式协议	能跑端到端工具闭环
评价与治理	四层评测、回归集、长上下文、安全、许可证	reward 不等于能力；生产边界	能提交可复现评测报告

MiniMind 教学知识依赖图

先建立协议与数据契约，再讨论模型、训练、后训练与系统交付

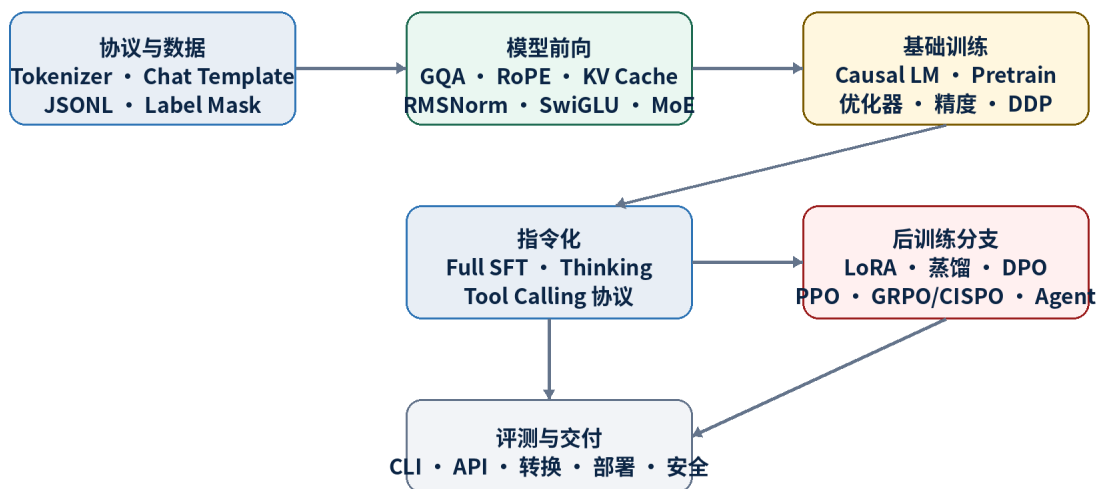


图 1 MiniMind 教学知识依赖图

课程版本建议

版本	建议时长	覆盖范围	适合场景
体验工作坊	8 学时	发布权重推理、Tokenizer/模板、模型结构概览、SFT 与评测	公开课、教师培训、技术分享
核心课程	24 学时	模块 1—10 + 模块 15；RL 只讲概念与 DPO	本科高年级、工程入门班
完整版	40 学时	15 个模块全部覆盖；含 PPO/GRPO/Agent 与部署	研究生、企业实训、科研组研讨

课时口径 本讲义中的“1 学时”按 45 分钟计算。带 GPU 的实验可放在课外异步完成，课堂只保留 smoke run、日志解读和验收。

40 学时完整版总表

模块	主题	学时	层级	最低产出
1	全局地图、环境与版本边界	2	核心	环境验收单
2	Tokenizer 与聊天协议	3	核心	模板/token 解剖图
3	Decoder Block 与张量流	4	核心	forward 形状记录
4	MoE、Causal Loss 与生成	3	核心	Dense/MoE 对照

模块	主题	学时	层级	最低产出
5	五类 Dataset 与 label mask	3	核心	mask 验收报告
6	训练公共机制与可复现性	3	核心	训练前检查表
7	Pretrain: 从随机参数续写	3	核心	smoke checkpoint
8	Full SFT: 把基座变成助手	3	核心	固定回归输出
9	LoRA 与知识蒸馏	3	核心/进阶	adapter/蒸馏对照
10	DPO 偏好对齐	2	核心	偏好 margin 分析
11	在线 RL 共用部件与 PPO	3	进阶	reward/KL/value 图
12	GRPO 与 CISPO	2	进阶	组内方差诊断
13	多轮 Agentic RL 与工具环境	2	进阶	轨迹与 mask 审计
14	推理、API、转换与部署	2	核心/进阶	端到端服务验收
15	评测、安全与综合项目	2	核心	评测报告与答辩

评价方案

评价项	权重	评价证据
课堂概念检查	15%	出口题、口头解释、板书补全
实验与可复现记录	30%	命令、配置、日志、权重命名、验收截图/文本
数据与协议审计	15%	token/template/mask 检查；异常样本说明
综合项目	30%	训练或适配、基线对照、回归评测、错误分析
安全与许可证说明	10%	模型/数据/依赖来源、风险与部署边界

评分原则 不奖励“跑出一个看起来不错的回答”；奖励可复现、可比较、能解释失败原因的证据链。

模块 1 全局地图、环境与版本边界

建议学时	课程层级	先修模块	最低课堂产出
2	核心	Python/命令行	一页环境验收单 + 训练路线图

学习目标

- 说清 MiniMind 的教学价值：用可训练的小模型观察完整 LLM 链路，而不是替代生产级模型。
- 区分 Pretrain→SFT 主线与 LoRA、蒸馏、DPO、PPO、GRPO/CISPO、Agent 等并列分支。
- 掌握项目根目录、trainer/、scripts/ 三种工作目录及相对路径后果。
- 记录源文件、代码提交、模型、数据与依赖版本，建立可复现实验基线。

1.1 用“小模型显微镜”理解全链路

MiniMind 默认 Dense 约 63.91M 参数，MoE 约 198.42M 总参数、每 token 激活路径约 64M。它的价值在于模型、数据、训练与服务代码足够短，学生可以把抽象概念落实到张量和文件。

教学主线只有两个必经阶段：随机初始化的 Causal LM 先做 Pretrain，再用对话数据做 Full SFT。其余后训练方法通常从同一个 SFT 基线分别实验，不能讲成必须依次串联的流水线。

概念与实现 课堂上用不同颜色标注“LLM 通用原理”和“本提交的实现选择”。例如 DPO 不需要奖励模型是通用性质；API 非流式 max_tokens 接线错误是版本实现问题。

1.2 仓库、产物与工作目录

正式训练产物主要有 out/ 推理权重、checkpoints/ 普通权重、以及带 optimizer/scheduler 等状态的 _resume 文件。自定义 save_dir 只改变输出目录，后续阶段仍可能固定从 ../out 加载；这类路径不一致必须在开课前演示。

工作位置	典型入口	默认相对路径
项目根目录	eval_llm.py	model/、out/
trainer/	train_*.py	../dataset、../model、../out
scripts/	API、Tool、WebUI、convert	按 scripts/ 计算上级目录

1.3 资源、依赖与许可边界

- requirements 没有自动安装 PyTorch；应先按本机 CUDA/CPU/MPS 选择合适 torch。
- 普通 Dense 训练与在线 RL 的显存需求完全不同；后者会同时驻留 policy、reference、critic/reward model。
- 仓库代码使用 Apache-2.0，不代表外部数据、奖励模型和依赖也自动获得同样许可。
- 加载 trust_remote_code 或 .pth 会执行/反序列化外部内容，只能使用可信来源与固定版本。

课堂流程

时间	教师活动	学生活动	验收证据
0—15 分	展示完整训练树，说明哪些是主线、哪些是分支	用箭头重画路线	路线图没有把所有后训练串联
15—35 分	带读目录与权重命名	定位模型、数据、trainer、scripts	能从产物名反推阶段/架构
35—65 分	演示环境检查与发布权重推理	记录版本、设备、速度和错误	环境验收单
65—90 分	用路径陷阱和许可问题做出口题	诊断两个失败场景	原因与修复均可复述

课堂实验：建立可复现实验头

实验目标：在不训练的前提下确认环境、tokenizer 和模型入口可用。

1. 记录 git commit、Python、torch、transformers、CUDA 与 GPU 名称。
2. 确认 model/tokenizer.json 与 tokenizer_config.json 存在。
3. 先用发布的 Transformers 权重跑固定三问，记录生成参数与耗时。
4. 把命令、版本、输出和异常写入同一份实验记录。

```
[bash]
python - <<'PY'
import torch, transformers
print('torch=', torch.__version__)
print('transformers=', transformers.__version__)
print('cuda=', torch.cuda.is_available())
print('gpu_count=', torch.cuda.device_count())
PY
```

通过标准

- 环境版本与代码提交已记录。
- 能解释本次加载的是原生 .pth 还是 Transformers 目录。
- 固定问题、生成参数、输出和耗时均有留档。

高频误区与教师追问

误区 把 DPO→PPO→GRPO→Agent 讲成四个必须连续执行的阶段。追问：如果 DPO 让通用能力回退，为什么还要把它作为 PPO 的唯一前置？

版本陷阱 README、教程与代码默认可能不同。追问学生：你引用的是“推荐值”还是“实际 argparse 默认值”？

形成性评价（含参考答案）

Q1. MiniMind 的最小必经训练路线是什么？

参考答案：Pretrain→Full SFT；其余后训练可从同一 SFT 基线独立比较。

Q2. 为什么相同命令在根目录和 trainer/ 运行结果不同？

参考答案：脚本使用相对路径；当前工作目录决定 tokenizer、数据与权重实际指向。

课后任务

- 1. 绘制自己的项目目录图，并为每个训练阶段写出输入文件、输出文件和验收命令。
- 2. 列出本地使用的模型、数据、奖励模型及其许可证/来源，标出尚未核验项。

模块 2 Tokenizer 与聊天协议

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 1	一份模板到 token 流的解剖报告

学习目标

- 解释 ByteLevel BPE 的词表构成、压缩率与可逆性。
- 准确区分 PAD/UNK、BOS、EOS、added token 与 special token。
- 逐角色说明 Chat Template 如何编码 system、user、assistant、tool、thinking 与 tool call。
- 区分 tokenizer 元数据长度、RoPE 缓冲长度和训练/截断长度。

2.1 6400 词 ByteLevel BPE

当前词表由 36 个协议/预留 token、256 个完整字节 token 与 6108 个 BPE 合并结果组成。ByteLevel 使普通 UTF-8 文本可以退化为字节序列，避免大规模 OOV，但小词表会让中文序列更长。

正式主线不重训 tokenizer，因为 token ID 是 embedding 行号。改词表等于重新定义模型输入空间，已有权重、模板、评测和部署配置都会失配。重训脚本只适合观察 BPE 训练过程。

ID 范围	含义	教学重点
0	PAD/UNK/end-of-text 共用	用 input_ids!=0 做 mask 会同时屏蔽真实 ID 0
1	< im_start > / BOS	tokenizer 不会自动添加

ID 范围	含义	教学重点
2	< im_end > / EOS	对话轮结束与生成停止
21—24	tool_call/tool_response 标签	added token，但不是 special
25—26	think 标签	decode(skip_special_tokens=True) 后仍保留

2.2 Chat Template 是“协议编译器”

模板把结构化 messages 编译为模型看到的单一 token 序列。工具定义写入 system 块；assistant 总有 think 区；连续 tool 消息被合并为新的 user turn 中的 tool_response。未知 role 没有渲染分支。

open_thinking=True 时 generation prompt 以 <think> 开头；关闭时先放一个完整空 think 块再让模型续写正文。这是同一模型的解码协议开关，不是切换另一套权重。

课堂演示 让学生把同一组 messages 分别用 open_thinking=False/True 渲染，然后只比较尾部差异。这样可以直接纠正“thinking 是另一模型”的误解。

2.3 三类长度必须分开

长度	当前值/来源	它能说明什么
tokenizer model_max_length	131072	元数据；不等于可靠上下文能力
模型 max_position_embeddings	32768	默认 RoPE cos/sin 缓冲上限
训练脚本 max_seq_len	阶段各自设置	本次样本截断与训练长度

课堂流程

时间	教师活动	学生活动	验收证据
0—25 分	用字节与 BPE merge 演示词表形成	手工编码中英文短句	能说明小词表取舍
25—55 分	拆解 0/1/2 与 Tool/Think token	预测 skip_special_tokens 结果	预测与实际一致
55—95 分	逐角色渲染 Chat Template	标注可训练/不可训练区	模板解剖图
95—120 分	对比三类长度并做错因讨论	判断 128K 宣称是否成立	回答包含训练证据要求

时间	教师活动	学生活动	验收证据
120— 135 分	出口题与实验收尾	提交 token 统计	结果可复现

课堂实验：渲染四类对话并检查 token

实验目标：观察普通对话、thinking、tool calling 与生成提示的精确文本边界。

1. 加载 ./model tokenizer，打印词表长度和关键 token ID。
2. 构造 system+user 对话，分别渲染 open_thinking 开/关。
3. 加入 tools、assistant tool_calls 和 tool response，观察角色重排。
4. 编码再解码 Tool/Think 标签，比较 skip_special_tokens=True/False。

```
[python]
from transformers import AutoTokenizer
tok = AutoTokenizer.from_pretrained('./model')
msgs = [
    {'role':'system','content':'你是严谨的助手。'},
    {'role':'user','content':'解释 GQA。'}
]
for flag in (False, True):
    text = tok.apply_chat_template(
        msgs, tokenize=False,
        add_generation_prompt=True,
        open_thinking=flag)
    ids = tok.encode(text, add_special_tokens=False)
    print(flag, len(ids), text[-120:])
```

通过标准

- `len(tokenizer)==6400`，关键 ID 与配置一致。
- 能指出 `open_thinking` 只改变 `generation prompt` 尾部。
- Tool/Think 标签在 `skip_special_tokens=True` 时仍可见。
- 报告中明确区分 131072、32768 与 `max_seq_len`。

高频误区与教师追问

误区 “tokenizer 写了 131072，所以模型支持 128K。”追问：位置编码、训练长度和长文任务评测证据在哪里？

协议风险 自定义模板后仍沿用旧 assistant marker 做 label mask，可能得到全 -100。追问：模板变化后要同步验证哪些边界字符串？

形成性评价（含参考答案）

Q1. 为什么 Tool/Think 标签不标为 special token?

参考答案：这样标签本身可参与 loss，且 skip_special_tokens 解码后仍可供推理代码解析。

Q2. 为什么重训 tokenizer 后不能直接加载旧权重?

参考答案：token ID 与 embedding 行的语义对应改变，旧权重中的行不再代表同一 token。

课后任务

1. 选择一段 200 字中文和一段等义英文，统计字符数、token 数与平均压缩率，并解释差异。
2. 设计一个含两次工具调用的 messages，手工画出渲染后的角色顺序，再与模板输出对照。

模块 3 Decoder Block 与张量流

建议学时	课程层级	先修模块	最低课堂产出
4	核心	模块 2、线性代数	带形状标注的 forward 与 cache 记录

学习目标

- 画出 Pre-Norm Decoder Block，并解释两次残差连接。
- 从 [B,T,768] 推导 Q/K/V、注意力分数与输出形状。
- 解释 GQA 相比 MHA 如何减少 KV Cache，以及 cache 中保存什么。
- 说明 RoPE 的作用、偶数 head_dim 约束和 YaRN 只解决位置外推的一部分。

MiniMind Decoder Block：课堂板书版

Pre-Norm + 残差；注意力层可使用 KV Cache，前馈层可切换 Dense/MoE

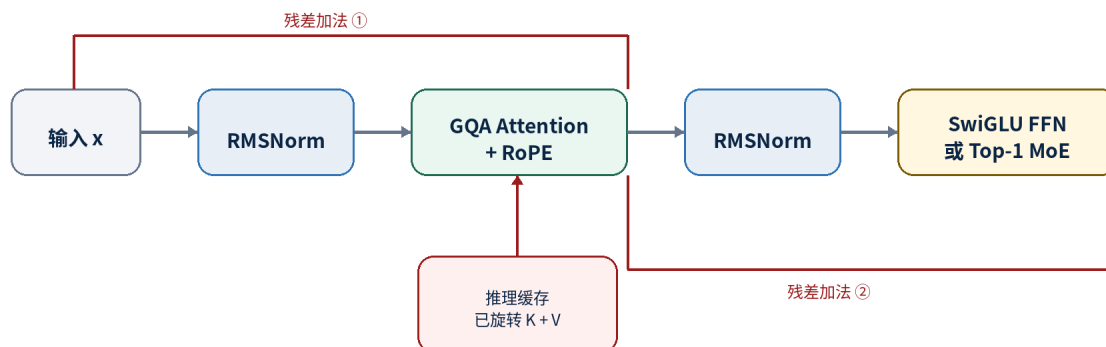


图 2 MiniMind Decoder Block 课堂板书版

3.1 默认模型配置与参数预算

配置	默认值	解释
hidden/layers	768 / 8	主干宽度与深度
Q heads / KV heads	8 / 4	GQA 2:1
head_dim	96	$768 \div 8$; RoPE 要求偶数
intermediate	2432	SwiGLU 中间维度
RMSNorm eps	$1e-6$	FP32 计算后转回输入 dtype
rope_theta	$1e6$	旋转位置编码频率基数
tied embeddings	true	输入 embedding 与 LM head 共权重

3.2 注意力、GQA 与形状推导

默认输入 x 为 $[B, T, 768]$ 。Q reshape 为 $[B, T, 8, 96]$ ，K/V 为 $[B, T, 4, 96]$ ；每个 KV head 复制两次服务 8 个 Q head。注意力分数为 $[B, 8, T, \text{total}_T]$ ，最终合并回 $[B, T, 768]$ 。

$$\text{Attention}(Q, K, V) = \text{softmax}((QK^T / \sqrt{d}) + M) V$$

M 同时承载因果遮罩与必要的 padding 遮罩。

3.3 RoPE 与 KV Cache

RoPE 把位置编码作为二维旋转施加到每个 Q/K 的偶数-奇数维对上，使点积包含相对位置信息。缓存保存的是已归一化、已旋转的 K，以及投影后的 V；增量解码只为新 token 计算 Q/K/V。

默认 SDPA 快路径要求序列长度大于 1、没有历史 cache、mask 为空或全 1，并且 PyTorch 提供 scaled_dot_product_attention。配置名 flash_attn=True 指 PyTorch SDPA，不代表安装了外部 flash-attn 包。

教师追问 如果缓存未旋转的 K，会发生什么？答案应指出：每次解码还需结合历史位置重新旋转，既增加计算，也容易错配 position。

3.4 RMSNorm、SwiGLU 与权重绑定

Dense 前馈使用 $\text{down_proj}(\text{silu}(\text{gate_proj}(x)) \times \text{up_proj}(x))$ ，所有投影无 bias。输入 embedding 与 LM head 绑权重可减少约 4.92M 参数，并让输入/输出 token 表示共享统计。

$$\text{RMSNorm}(x) = x / \sqrt{(\text{mean}(x^2) + \epsilon)} \cdot w$$

实现先转 FP32 计算，再转回输入 dtype。

课堂流程

时间	教师活动	学生活动	验收证据
0—35 分	从 config 推导各张量形状	完成形状填空	形状无遗漏
35—80 分	板书 Pre-Norm、残差与 SwiGLU	解释梯度路径	能区分 Norm 位置
80—125 分	演示 GQA 与 cache 首轮/增量解码	记录 cache 形状变化	cache 长度按 token 增长
125—160 分	讲 RoPE/YaRN 与 SDPA 快慢路径	判断四种场景走哪条路径	理由引用条件
160—180 分	小配置 forward 实验	提交参数量与输出形状	CE/aux 均有限

课堂实验：小配置 forward 与 cache 观察

实验目标：用可打印的小模型验证形状，而不是只记架构名词。

1. 构造 hidden=128、layers=2、Q=4、KV=2 的小模型。
2. 对 [2,16] token IDs 做带 labels 的 forward，记录 logits/loss。
3. 启用 use_cache，先输入 8 个 token，再输入 1 个 token，比较每层 K/V 长度。
4. 把 head_dim 改成奇数可触发的配置，解释为什么 RoPE 维度不匹配。

```
[python]
import torch
from model.model_minimind import MiniMindConfig, MiniMindForCausalLM
cfg = MiniMindConfig(hidden_size=128, num_hidden_layers=2,
                      num_attention_heads=4, num_key_value_heads=2,
                      vocab_size=6400, max_position_embeddings=128)
m = MiniMindForCausalLM(cfg).eval()
ids = torch.randint(3, 6400, (2, 16))
out = m(input_ids=ids, labels=ids, use_cache=True)
print(out.logits.shape, float(out.loss))
for k, v in out.past_key_values:
    print(k.shape, v.shape)
```

通过标准

- logits 形状为 [2,16,6400]。
- 每层 K/V 的 head 数是 2，而不是 Q 的 4。
- 能说明 cache 中 K 已应用 RoPE、V 未旋转。
- 能用条件而非名称解释 SDPA 快路径。

高频误区与教师追问

误区 把 GQA 理解为“减少 Q head”。实际 Q head 仍是 8，减少的是独立 K/V head。

长上下文 YaRN 让位置编码可外推，不会自动补上长文训练数据、检索能力和可靠性。

形成性评价（含参考答案）

Q1. 默认 GQA 的 KV Cache 相比 8 个 KV head 约减少多少？

参考答案：K/V head 从 8 降到 4，其他条件不变时缓存约减半。

Q2. 为什么缓存 K 而不是每步重算历史 K？

参考答案：历史 token 的投影与位置不变；缓存避免每个新 token 重复计算整段历史。

课后任务

1. 推导 $B=2$ 、 $T=128$ 时 Q/K/V 与 scores 的元素数量，并比较 MHA 与 GQA 的 cache。
2. 画一张首轮 prefill 与单 token decode 的计算差异图，标出哪些张量可复用。

模块 4 MoE、Causal Loss 与自定义生成

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 3	Dense/MoE 目标函数与生成策略对照

学习目标

- 解释 MoE 总参数、激活参数、router 与专家负载之间的关系。
- 正确组合语言模型 CE 与 router auxiliary loss。
- 说明模型内部 next-token shift，避免 Dataset 再次 shift。
- 比较 temperature、top-k、top-p、greedy、repetition penalty 与 KV cache。

4.1 Top-1 MoE 的真实计算

每个 token 经 gate 得到 4 个专家概率，默认只选 1 个完整 SwiGLU 专家。总参数约 198.42M，但每 token 只激活约 63.94M，因此写作 198M-A64M。它降低的是每 token 计算量，不是权重存储量。

Top-1 且选中概率重新归一化后，唯一权重接近 1，router 的主要训练信号来自负载均衡辅助损失。训练脚本必须显式把 aux_loss 加到 CE；模型 output.loss 本身只含 CE。

$$L_{\text{total}} = L_{\text{CE}} + \lambda \cdot \sum_e \text{load_e} \cdot \text{mean_prob_e} \cdot E$$

当前 $\lambda=5 \times 10^{-4}$, $E=4$; 具体实现应以提交版本为准。

4.2 Causal LM 与 shift

模型用 `logits[... :-1, :]` 预测 `labels[..., 1:]`, 并忽略 -100。Pretrain/SFT Dataset 应返回等长 `input_ids` 与 `labels`, 不要在数据层再移一位; DPO 不走模型 CE, 因此才会在 Dataset 中提前构造 `x/y`。

$$L_{\text{CE}} = - \sum_t \log p_{\theta}(x_t | x_{<t}), \text{ labels}=-100 \text{ 的位置不计入}$$

典型故障 SFT 样本被截断到没有任何 assistant token 时, labels 可能全是 -100, mean CE 会出现 NaN。训练前必须统计 active labels。

4.3 生成不是“模型再 forward 一次”

参数	作用	常见误解
temperature	缩放 logits	temperature=0 会除零; greedy 用 do_sample=False
top-k	只保留最高 k 个 token	k 不能超过 vocab
top-p	保留累计概率质量	不同后端默认值可能不一致
repetition penalty	抑制已出现 token	不能替代重复率评测
use_cache	复用历史 K/V	不等于允许无限长度

课堂流程

时间	教师活动	学生活动	验收证据
0—35 分	用 token→router→expert 图讲 MoE	计算总/激活参数含义	能解释 A64M
35—70 分	演示模型内 shift 与 -100	手工对齐 6-token 样本	预测目标正确
70—110 分	对比 greedy 与采样参数	固定 prompt 多次生成	记录差异而非挑样例
110—135 分	Dense/MoE 小实验与出口题	检查 CE/aux	总目标写法正确

课堂实验：Dense/MoE 与解码策略对照

实验目标：验证 aux loss 与采样策略对结果的独立影响。

1. 用相同小配置分别实例化 Dense 和 MoE。
2. 同一 batch 计算 CE、aux 与 total，确认 Dense aux=0。
3. 在同一 checkpoint 上用 greedy 与三组 temperature/top-p 生成。
4. 固定 prompt 与随机种子，记录输出长度、重复和 EOS 是否出现。

```
[python]
out = model(input_ids=ids, labels=ids)
train_loss = out.loss + out.aux_loss
print('ce=', float(out.loss),
      'aux=', float(out.aux_loss),
      'total=', float(train_loss))
```

通过标准

- Dense aux 为 0，MoE aux 非负。
- 训练目标显式包含 aux。
- greedy 使用 do_sample=False，而非 temperature=0。
- 报告包含采样参数和种子。

高频误区与教师追问

误区 “MoE 有 198M 参数，所以每个 token 都计算 198M。”应区分存储参数、激活路径和 router 开销。

实现边界 自定义 generate 不是完整 Hugging Face Generation API；没有 beam search、stop string、多 EOS 等能力。

形成性评价（含参考答案）

Q1. 为什么 HF Trainer 直接读取 output.loss 可能训练不好 MoE？

参考答案：当前 output.loss 不含 router aux；必须自行把 aux 加入目标。

Q2. Dataset 已经把 labels 左移一位会怎样？

参考答案：模型内部还会再 shift，导致目标错一位。

课后任务

1. 写出一个 5-token 序列的输入、模型内部 shift 后 logits/label 配对和忽略位置。
2. 为生成实验设计一个小表，至少记录 seed、temperature、top-p、长度、重复率和是否 EOS。

模块 5 五类 Dataset 与 label mask

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 2、4	覆盖五类样本的 mask 验收报告

学习目标

- 识别 Pretrain、SFT、DPO、RLAIF、Agent 五种数据契约。
- 解释 assistant-only mask、右 padding、截断与模型内 shift 的组合。
- 能发现非法 JSON、空对话、错误占位、全 -100 与 prompt 不一致。
- 为普通、多轮、reasoning、tool call 和长 prompt 建立最小验收集。

5.1 五种输入不是同一种 JSONL

Dataset	输入核心	输出/训练信号
Pretrain	{text}	BOS+正文+EOS；等长 labels
SFT	conversations	仅 assistant 片段参与 CE
DPO	chosen/rejected	两边已 shift 的 x/y/mask
RLAIF	prompt+末尾空 assistant	字符串 prompt；在线采样
Agent	tools+gt+末尾空 assistant	messages/tools/gt；多轮轨迹

5.2 SFT mask 是课程最重要的验收点

system、user、tool response 与 padding 的 labels 设为 -100；assistant marker 后的 think、正文、tool_call、结束 token 均训练。边界依赖模板中的精确 assistant 开始与 im_end+换行。模板一改，mask 代码必须一起回归。

右截断可能把 assistant 完全切掉；长 user prompt、工具 schema 和多轮历史都是高风险样本。应把 active label 数量作为数据质量指标，而不是等到 loss NaN 才排查。

$$\text{active_ratio} = \text{count}(\text{labels} \neq -100) / \text{sequence_length}$$

建议同时报告 0 分位、P5、P50、P95；任何 active_ratio=0 都应阻止正式训练。

5.3 在线 RL 数据的“末尾空 assistant”

RLAIFDataset 与 AgentRLDataset 都会无条件丢掉 conversations 的最后一条，默认它是等待模型生成的空 assistant。若数据末尾不是占位，真实消息会被静默删除。Agent 的 gt 实践中必须是 list，tools 通常挂在 system 消息上。

教学设计 故意准备一条末尾不是 assistant 的坏样本，让学生通过渲染 prompt 发现最后一条内容消失。

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	对比五类 JSONL 契约	给样本分类	分类与理由
30—70 分	逐 token 展示 SFT mask	给 token 着色	只训练 assistant
70—105 分	制造截断、坏 JSON、错占位	运行数据审计	发现并定位异常
105—135 分	DPO/RLAIF/Agent 特例讲解	解释各自为何不同	输出字段正确

课堂实验：主动标签解码与异常样本注入

实验目标：让 labels 变成可读文本，验证“模型究竟被要求学习什么”。

1. 从普通、多轮、reasoning、tool call 数据各抽一条。
2. 打印有效 input_ids 文本与 labels!=-100 的目标文本。
3. 构造超长 user prompt，观察 active labels 是否归零。
4. 对 DPO 检查 chosen/rejected prompt 是否相同、两边 mask 是否非空。

```
[python]
ids, labels = ds[0]
active = labels.ne(-100)
print('active=', int(active.sum()), '/', len(labels))
print('full=', tok.decode(ids[ids.ne(tok.pad_token_id)]))
print('target=', tok.decode(labels[active]))
assert active.any(), 'all labels are -100'
```

通过标准

- 普通、多轮、reasoning、tool call 均有样本。
- target 解码中不含 user/system/tool response 正文。
- 没有 active labels 为 0 的样本。

- DPO chosen/rejected 的 prompt 语义一致。

高频误区与教师追问

误区 “有 padding 就必须传 attention_mask，否则答案错误。”当前右 padding 与因果遮罩可防有效 token 看未来 pad，但 padding 仍浪费算力；要讨论的是正确性与效率两个层面。

数据风险 JSONL 每条完整样本必须是一个物理行。把文档中为阅读而换行的 JSON 直接保存会逐行 json.loads 失败。

形成性评价（含参考答案）

Q1. 为什么 SFT 要屏蔽 user/system token 的 loss?

参考答案：目标是学习在给定上下文下生成 assistant，而不是复述用户或系统文本。

Q2. RLAIFFDataset 为什么会丢最后一条消息?

参考答案：约定最后一条是空 assistant 占位；前面的消息才构成在线生成 prompt。

课后任务

1. 写一个数据审计脚本，输出样本数、坏 JSON 数、空对话数、active label 分位数和最长 prompt。
2. 为 Tool Calling 样本定义 schema 校验规则，至少检查 tool name、arguments 类型和必填字段。

模块 6 训练公共机制与可复现性

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 5	训练前检查表 + checkpoint/resume 说明

学习目标

- 计算有效全局 batch，解释 GPU 数、microbatch 与累积步数的联动。
- 说明 AdamW、余弦衰减、混合精度、GradScaler 与梯度裁剪的作用。
- 区分 out 权重、普通 checkpoint 与 resume checkpoint。
- 认识 DDP 与在线 RL 多卡更新的适用边界，避免把“能运行”当“同步正确”。

6.1 有效 batch 与优化条件

增加 GPU 数但保持每卡 batch、累积步数和学习率不变，会改变全局 batch 与每个 epoch 的优化步数。任何速度比较都应同时报告 tokens/step、global batch 与总 token 数。

$$\text{global_batch} \approx \text{micro_batch} \times \text{world_size} \times \text{accumulation_steps}$$

适用于 Pretrain/SFT/LoRA/蒸馏/DPO；PPO/GRPO/Agent 还要考虑 rollout 与多次更新。

6.2 优化器、学习率与精度

- 所有脚本使用 AdamW；未显式设置的 betas、eps、weight_decay 继承 PyTorch 默认。
- 监督/偏好脚本使用无 warmup 的余弦衰减，从约 100% 降至 10%。
- CUDA 默认 BF16 autocast；CPU/MPS 不会因为 --dtype bfloat16 自动进入 BF16。
- 仅精确传 --dtype float16 时，前五类脚本启用 GradScaler；在线 RL 没有 FP16 scaler。
- 默认梯度裁剪 1.0；它限制爆炸梯度，不会修复错误数据或 NaN 来源。

6.3 保存、恢复与近似可复现

resume 通过跳过已完成 batch 继续，但不保存完整 Python/NumPy/Torch RNG 状态；Dataset 里的随机 system/空 think 处理也难以逐 bit 复现。更换 world size 后 step 只做整数近似。

文件	主要内容	用途
out/<name>.pth	CPU FP16 state_dict	推理或下一阶段加载
checkpoints/<name>.pth	模型权重	阶段中间模型
*_resume.pth	模型+optimizer+epoch/step+附加状态	继续训练

6.4 DDP 与日志边界

普通训练通过 DDP wrapper forward；GRPO、Agent 以及 PPO 的部分更新显式取 .module 后 forward，需要单独验证多卡梯度同步。teacher/reference/reward model 在每张卡各复制一份，不做分片。参数 --use_wandb 在当前代码里实际导入 swanlab。

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	用三个配置计算 global batch	完成计算与学习率讨论	数值与解释一致
30—65 分	讲优化器、余弦与精度	判断 BF16/FP16/CPU 场景	能指出 scaler 条件

时间	教师活动	学生活动	验收证据
65—100 分	拆 resume 文件并演示中断恢复	定位状态字段	知道哪些状态没保存
100—135 分	DDP 边界与日志阅读	设计单双卡对照	同步验证方案

课堂实验：训练前检查与近似 resume

实验目标：在正式烧 GPU 前验证 tokenizer、数据、权重命名、batch 和保存路径。

1. 验证 tokenizer 6400、BOS/EOS/PAD ID。
2. 计算并记录 global batch 和每 epoch 预计 optimizer steps。
3. 对 256 条 smoke 数据设置短 save_interval，产生 resume checkpoint。
4. 中断后用相同参数恢复，核对 step、loss 与学习率是否合理连续。

```
[python]
micro = 4
world = 2
accum = 8
print('global_batch=', micro * world * accum)
# 同时记录: dataset_size、drop_last、每 epoch batch/step 数
```

通过标准

- global batch 计算正确且写入实验记录。
- 保存路径与下一阶段加载路径一致。
- resume 从合理 step 继续，前几步 loss/lr 无突变。
- 报告明确写“近似续跑”，不宣称逐 bit 恢复。

高频误区与教师追问

误区 只看每卡 batch，忽略 world size 与 accumulation。追问：双卡后一个 epoch 的 optimizer step 数和每步样本数怎么变？

工程风险 save_interval 不与 accumulation 对齐时，可能保存到尚未 optimizer.step 的累积中间状态；未提交梯度不会进入 resume。

形成性评价（含参考答案）

Q1. Pretrain 默认单卡有效 batch 是多少？

参考答案：32×1×8=256 条；序列长度仍需另报。

Q2. 为什么 resume 后不能保证完全相同？

参考答案：未保存完整 RNG，随机数据处理会变化，world size/累积中间梯度也可能不同。

课后任务

1. 为自己的设备写一张 batch/显存/吞吐实验表，至少测试两组 microbatch 与 accumulation 组合。
2. 设计一个验证 DDP 更新等价性的最小实验：固定 batch、seed，比较单卡与双卡一步后的参数差异。

模块 7 Pretrain：从随机参数学习续写

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 4—6	可加载的 smoke checkpoint + 训练曲线解释

学习目标

- 把 Pretrain 解释为 next-token prediction，而不是聊天能力训练。
- 掌握 smoke→正式训练的逐级放大方法。
- 能读懂默认配置、README 推荐与硬件现实之间的差异。
- 用有限 loss、可加载产物、续写质量和数据规模共同验收。

7.1 目标与数据流

PretrainDataset 把每条 text 截到 max_length-2，再手工加 BOS/EOS、右 padding，并把 ID 0 的 labels 屏蔽。模型学习 BOS→首 token、正文→下一 token、最后正文→EOS。短样本没有 packing，会浪费 padding 计算。

7.2 版本默认与实验变量

参数	当前默认	教学说明
epochs	2	成本表常按 1 epoch，不能混用
batch/accum	32 / 8	单卡 global batch 256
learning rate	5e-4	无 warmup 的余弦衰减
max_seq_len	340	README 对 mini 另建议约 768

参数	当前默认	教学说明
from_weight	none	随机初始化

实验原则 先证明接线正确，再扩大数据/长度/模型。没有 --max_steps 时，复制少量 JSONL 到独立 smoke 文件，不修改原始数据。

7.3 验收不是只看 loss

- loss 有限且整体下降；同时报告有效 token 数与学习率。
- 权重文件能由与训练一致的 config 加载。
- Pretrain 输出可形成局部续写，但不要求遵循聊天指令。
- MoE 除 CE 还要监控 aux 与专家 token count，否则看不到专家坍塌。
- 同一数据/总 token 下再比较 Dense 与 MoE，避免把训练预算差异当架构差异。

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	从文本样本推导 target	手工完成 BOS/EOS/shift	目标序列正确
30—60 分	对比默认与 README 推荐	估算长度平方开销	能解释为何先 340
60—105 分	启动 smoke 并读日志	记录 loss/lr/step	接线无 NaN
105—135 分	加载产物并做前缀续写	区分续写与聊天	验收结论合理

课堂实验：256 条数据的 Pretrain smoke

实验目标：快速验证数据→模型→优化器→保存→加载全链路。

1. 从 pretrain_t2t_mini.jsonl 复制前 256 条到新文件。
2. 使用 1 epoch、batch=4、accum=1、workers=0、seq=128。
3. 每步打印日志，并设置足够短的 save_interval。
4. 用 eval_llm.py 加载 smoke 权重，只判断接线，不评价知识能力。

```
[bash]
cd trainer
python train_pretrain.py \
  --data_path ../dataset/pretrain_smoke.jsonl \
  --epochs 1 --batch_size 4 --accumulation_steps 1 \
  --num_workers 0 --max_seq_len 128 \
  --log_interval 1 --save_interval 20 \
  --save_weight pretrain_smoke
```

通过标准

- loss 为有限值，且没有 all -100 / shape 错误。
- out/pretrain_smoke_*.pth 存在并可加载。
- 实验记录写明 smoke 不代表模型质量。
- 正式训练方案注明数据、epoch、长度、global batch 与预算。

高频误区与教师追问

误区 用聊天指令服从去验收 Pretrain。正确问题应是：模型是否学会局部续写与 token 统计，而不是扮演助手。

成本陷阱 “两小时/三元”是特定硬件、数据、epoch 的估计，不等于默认 2 epochs 在任何设备上的承诺。

形成性评价（含参考答案）

Q1. 为什么 PretrainDataset 要手工加 BOS/EOS?

参考答案：tokenizer 配置 add_bos_token/add_eos_token 均为 false。

Q2. 序列长度从 340 增到 680，注意力核心开销近似如何变化?

参考答案：其他条件不变时约按 T^2 ，从 1 倍增到 4 倍。

课后任务

1. 比较 seq=128/256 两个 smoke 的 tokens/s、显存和 loss，说明比较是否公平。
2. 设计一个 Pretrain 验收表，至少包含有限 loss、保存加载、续写、吞吐和错误样本。

模块 8 Full SFT：把基座变成聊天助手

建议学时	课程层级	先修模块	最低课堂产出
3	核心	模块 7	SFT 权重 + 50—200 条固定回归集

学习目标

- 解释 SFT 如何在保持语言能力的同时学习角色、指令、thinking 与工具协议。
- 掌握从 Pretrain 权重、SFT 数据到 full_sft 产物的接线。
- 建立协议、能力和错误类型三层验收，而不是挑选漂亮回答。
- 认识 64M 模型的容量上限与事实可靠性边界。

8.1 SFT 学的是条件行为

同样使用 Causal LM CE，但 labels 只保留 assistant 输出。模型在 system/user/tool 上下文条件下学习生成 think、正文、tool_call 与结束标记。当前 sft_t2t_mini 已混合 reasoning 和 Tool Calling 数据，通常不需要单独的工具 SFT 阶段。

8.2 当前默认与前置权重

参数	默认值	风险点
from_weight	pretrain	必须存在同 hidden/layers/Dense-MoE 权重
epochs/batch	2 / 16	每卡 microbatch
learning rate	1e-5	比 Pretrain 小
max_seq_len	768	长 user/tool schema 可能挤掉 assistant
output	full_sft	后训练常从此基线分支

8.3 回归集优先于主观印象

- 协议：assistant 边界、thinking 闭合、tool JSON 合法。
- 能力：中英文、知识、推理、代码、多轮、目标领域。
- 稳定性：多 seed 采样、重复率、长度、EOS、格式失败率。
- 安全：事实幻觉、危险工具、提示注入和超长输入。

教师口径 “输出流畅”只证明局部语言建模，不证明事实正确。小模型的错误与重复应当成为分析材料，而不是被隐藏。

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	对比 Pretrain/SFT labels	解释行为变化来源	能区分知识与协议

时间	教师活动	学生活动	验收证据
30—65 分	核对前置权重与默认配置	完成训练前检查	架构/路径一致
65—105 分	跑短 SFT 并观察 target	记录 loss 与样本	无全 -100
105—135 分	固定回归集对照	对错误分类	至少三类失败

课堂实验：SFT 前后问题对照

实验目标：用同一组 prompt 比较 Pretrain 与 Full SFT 的行为变化。

1. 训练前抽查普通、多轮、reasoning、tool 样本的 active labels。
2. 运行 Full SFT 或小数据接线实验，保存独立权重名。
3. 固定 tokenizer、模板、open_thinking、seed 与生成参数。
4. 对 Pretrain/SFT 输出按协议、正确性、重复、长度分类。

```
[bash]
cd trainer
python train_full_sft.py

# 根目录验收
python eval_llm.py --weight full_sft --max_new_tokens 512
python eval_llm.py --weight full_sft --open_thinking 1 --max_new_tokens 512
```

通过标准

- full_sft_768[_moe].pth 可加载。
- 相同 prompt 的生成条件保持一致。
- 回归集含 thinking/tool/multiturn/长输入。
- 报告至少列出三类错误及频率。

高频误区与教师追问

误区 把 SFT 后的流畅回答当成知识可靠。追问：同一事实题多 seed 的一致率是多少？

协议风险 thinking 与 Tool Calling 同时开启可能不稳定；不要只测单独能力，要测联合样本。

形成性评价（含参考答案）

Q1. SFT 为什么使用更小学习率？

参考答案：它从已学语言规律的 Pretrain 权重继续，目标是调整行为，过大学习率更易遗忘。

Q2. 如何证明 SFT 改善来自训练而不是采样幸运？

参考答案：固定任务和解码条件，进行多样本/多 seed 统计并比较错误率。

课后任务

1. 建立 50—200 条回归集，并为每条定义可判定的预期或错误类型。
2. 比较 open_thinking 开/关的格式成功率、答案正确率与平均长度。

模块 9 LoRA 与白盒知识蒸馏

建议学时	课程层级	先修模块	最低课堂产出
3	核心/进阶	模块 8	adapter 验证 + 教师/学生蒸馏设计

学习目标

- 推导仓库手写 LoRA 的注入位置、参数量与初始化性质。
- 区分原生 LoRA adapter 与 PEFT 格式，并掌握加载/合并边界。
- 解释白盒蒸馏的 CE+KL 目标、温度和教师冻结。
- 用目标域改善与通用回归共同评估适配/蒸馏。

9.1 MiniMind 手写 LoRA

LoRA 在原线性层旁加 $B(A(x))$ 。A 正态初始化、B 全零，所以注入瞬间输出不变。当前仅注入 $\text{in_features} == \text{out_features}$ 的方形 Linear；默认模型实际命中每层 q_proj 与 o_proj ，不含 K/V、FFN、专家、router 或 LM head。

$$y = Wx + BAx, \text{ rank}=16, \text{ 参数量}=16 \times (768 \times 16 + 16 \times 768) = 393,216$$

当前实现无 alpha/rank 缩放、无 LoRA dropout；rank 未写入 adapter 元数据。

9.2 加载与合并

- `apply_lora` 只注入，不负责冻结；训练脚本按参数名冻结基模。
- 加载 adapter 前必须以相同 rank 注入，否则可能没有匹配 key。
- 基础权重、hidden/layers 与 Dense/MoE 必须与训练时一致。
- 合并计算 $W_{\text{merged}} = W_{\text{base}} + B @ A$ ；FP16 保存会引入小的舍入差异。
- `convert_model.py` 没有 `argparse`；默认执行不等于 LoRA merge，需要显式审查底部函数。

9.3 白盒蒸馏

默认学生是 Dense full_sft，教师是 MoE full_sft。只完成 Dense 主线时会缺教师权重。学生/教师可用不同架构，但必须使用同一 tokenizer/vocab；更强教师还会显著增加显存。

$$L = \alpha \cdot \text{CE}(y, \text{student}) + (1-\alpha) \cdot T^2 \cdot \text{KL}(\text{teacher}/T \parallel \text{student}/T)$$

KL 只在 assistant 非 -100 token 上计算；教师冻结并 no_grad。

比较设计 LoRA 问“少量可训练参数能否学到目标域行为”；蒸馏问“学生能否逼近更强教师的分布”。两者的基线、资源和失败模式不同。

课堂流程

时间	教师活动	学生活动	验收证据
0—40 分	推导 LoRA 参数量与命中层	在模块树中标注 adapter	命中 Q/O
40—75 分	演示注入、保存、加载、合并	检查 key 与输出	adapter 只有 .lora.
75—110 分	推导 CE+KL 蒸馏	解释 α 、T	目标函数正确
110—135 分	设计两类对照实验	提交评价矩阵	含通用回归

课堂实验：adapter 完整性与合并等价性

实验目标：证明 adapter 结构正确，并量化 merge 前后差异。

1. 对 full_sft 注入 LoRA，确认未训练时输出与基模相同。
2. 训练小型垂域 adapter，检查保存文件只含 .lora. key。
3. 分别运行基模、基模+adapter、merged 权重。
4. 在目标域和通用集上比较，记录第一处 token 分叉与数值误差。

```
[python]
import torch
d = torch.load('out/lora_medical_768.pth', map_location='cpu')
assert d and all('.lora.' in k for k in d)
print('adapter tensors=', len(d))
print(*list(d)[:8], sep='\n')
```

通过标准

- adapter 文件仅含 LoRA key。
- B=0 的初始注入不改变输出。
- merge 前后结果近似，差异可由 dtype/舍入解释。

- 目标域改善同时报告通用集回退。

高频误区与教师追问

误区 把仓库 LoRA 当作 PEFT adapter 直接互换。两者注入层、缩放、元数据和文件格式都可能不同。

蒸馏门槛 默认教师是 MoE full_sft；只训练 Dense 后直接运行会缺文件。

形成性评价（含参考答案）

Q1. 为什么 LoRA 注入瞬间输出不变？

参考答案：B 全零，因此 $B(A(x))=0$ 。

Q2. 蒸馏为什么乘 T^2 ？

参考答案：温度缩小 softmax 梯度尺度， T^2 用于补偿，使不同温度下梯度量级更可比。

课后任务

1. 统计当前模型所有 Linear 的形状，列出哪些会被方形层规则注入、哪些不会。
2. 设计一个蒸馏实验表，包含教师质量、学生基线、CE/KL、显存和固定回归集。

模块 10 DPO 偏好对齐

建议学时	课程层级	先修模块	最低课堂产出
2	核心	模块 5、8	偏好数据审计 + margin/回归分析

学习目标

- 解释 DPO 如何用 policy/reference 的相对偏好差直接优化，无需 reward model/critic。
- 理解 chosen/rejected 序列 logprob 求和与长度偏差。
- 审计 prompt 一致性、偏好方向和 assistant mask。
- 用偏好 margin 与通用回归共同判断训练是否成功。

10.1 目标函数

policy 与 reference 都从同一 full_sft 初始化；policy 训练、reference 冻结。DPO 优化的是相对偏好，不在线探索，也不保证事实知识或推理能力提高。

$$L_DPO = -\log \sigma\{\beta[(\log n(y^+|x) - \log n(y^-|x)) - (\log n_{\text{ref}}(y^+|x) - \log n_{\text{ref}}(y^-|x))]\}$$

当前实现对 assistant token logprob 求和，不做长度归一化。

10.2 数据与超参数

项目	当前默认	验收重点
data	dpo.jsonl	chosen/rejected prompt 应相同
lr	4e-8	非常小，防止灾难遗忘
beta	0.15	偏离 reference 的强度尺度
max_seq_len	1024	两边 mask 必须非空
output	dpo_768.pth	与 full_sft 固定条件比较

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	从二分类偏好推导 DPO	标注四个 logprob	公式方向正确
30—60 分	审计 chosen/rejected 与 mask	找出坏样本	提示一致、方向正确
60—90 分	读 loss/margin 与回归	判断三种失败	结论有证据

课堂实验：DPO 数据与 margin 审计

实验目标：在训练前证明偏好对有效，训练后证明改善不是通用能力坍塌。

1. 抽样人工确认 prompt 相同、chosen 真优于 rejected。
2. 解码两边 assistant mask，统计有效 token 数与长度差。
3. 训练短实验并记录 chosen/rejected policy 与 reference margin。
4. 在偏好集、事实集和格式集上对 full_sft 与 dpo 做固定比较。

```
[python]
# 伪代码：每个 batch 至少记录
policy_margin = logp_pi_chosen - logp_pi_rejected
ref_margin = logp_ref_chosen - logp_ref_rejected
implicit_reward_margin = policy_margin - ref_margin
```

通过标准

- chosen/rejected prompt 一致且方向无误。

- 两边 assistant mask 非空。
- implicit reward margin 的变化可解释。
- 通用回归未被偏好指标掩盖。

高频误区与教师追问

误区 chosen 胜出不等于知识更正确；偏好数据可能只奖励语气、长度或格式。

长度偏差 序列 logprob 求和会随长度累积，长短差异可能影响 preference score。

形成性评价（含参考答案）

Q1. DPO 为什么不需要单独训练 reward model?

参考答案：偏好对和 reference 共同定义隐式奖励差，目标可直接写成可微的分类损失。

Q2. 训练开始时 policy/reference 相同，DPO logit 大致是多少？

参考答案：两者相对 margin 差接近 0，sigmoid 输入接近 0。

课后任务

1. 设计 20 对偏好数据，说明每对的偏好准则，并检查长度是否成为混杂变量。
2. 为 DPO 报告设计四列：偏好成功、事实正确、格式合法、通用回归。

模块 11 在线 RL 共用部件与 PPO

建议学时	课程层级	先修模块	最低课堂产出
3	进阶	模块 8、概率/强化学习基础	reward/KL/value/clip 联合诊断图表

学习目标

- 解释 rollout、old policy logprob、reference、reward model 与 critic 的职责。
- 推导 GAE、PPO ratio、clip、value loss 与 KL 正则的基本作用。
- 识别序列级 reward 放到末 token、reward shaping 与 reward hacking。
- 能用多指标而非 reward 单值判断 PPO 训练。

11.1 RLAIIF 与奖励组成

仓库把奖励模型、规则函数、GT 校验和环境反馈都广义归为自动反馈。普通 PPO/GRPO 默认使用 InternLM2-1.8B Reward Model 分数，并叠加长度、thinking 格式和 3-gram 重复规则。规则奖励会改变模型行为，也会提供被利用的漏洞。

安全边界 reward model 通过 `trust_remote_code=True` 加载；课堂环境应使用可信仓库与固定 revision。

11.2 PPO 的五个驻留组件

Critic 仍保留完整 CausalLM backbone 和未使用 LM head，再加 value head，因此显存高于“只多一个线性层”的直觉。当前实现还对 final hidden 做了第二次 final RMSNorm，这是仓库细节，不是 PPO 必需。

组件	是否训练	职责
Actor/Policy	是	生成并接受策略梯度
Reference	否	约束偏离，计算 KL
Critic+Value Head	是	估计 token value 与 advantage
Reward Model	否	给序列级分数
Rollout Engine	否	生成 completion 与 old logprob

11.3 PPO 数学骨架

外部序列奖励只放到 completion 最后一个有效 token，然后通过 GAE 传播；advantage 在有效回答 token 上归一化。同一 rollout 默认做两轮 PPO update，approximate KL 过大时提前停止。

$$r_t(\theta) = \exp(\log \theta(a_t | s_t) - \log \theta_{old}(a_t | s_t)); \quad L_{clip} = -E[\min(r_t A_t, \text{clip}(r_t, 1-\epsilon, 1+\epsilon) A_t)]$$

默认 $\epsilon=0.2$ ；GAE $\gamma=1.0$ 、 $\lambda=0.95$ ；value loss 与 reference KL 共同加入。

11.4 Rollout 后端差异

- Torch 后端从当前训练模型生成，并重新算 per-token old logprob。
- SGLang 后端通过 HTTP 获取生成与 logprob，采样参数不完全等同 Torch。
- SGLang 权重默认按 save_interval 才同步，期间 rollout policy 可能滞后。
- 缺失 logprob 左侧补 0 会污染 importance ratio；应把缺失视为错误而非静默容错。

课堂流程

时间	教师活动	学生活动	验收证据
0—35 分	画 PPO 五模型/引擎关系	标注冻结/训练	驻留图正确
35—75 分	推导 ratio/clip/GAE/KL	用数值例判断 clip	方向与截断正确
75—105 分	拆 reward 组成与 hacking	设计反例	指出可利用规则
105—135 分	读 PPO 日志并诊断	联合判断 reward/value/KL	不单看 reward

课堂实验：小批 PPO 指标联动

实验目标：观察 reward 上升时 KL、value、长度与重复是否共同健康。

1. 使用 debug_mode、小 batch、短 generation 先跑接线。
2. 补充记录 reward variance、回答长度与重复率。
3. 对每步记录 actor loss、value loss、approx KL、reference KL、clip fraction。
4. 抽样保存生成文本，标记凑长度、重复、think 标签投机。

```
[bash]
cd trainer
python train_ppo.py --debug_mode \
  --batch_size 2 --max_gen_len 256 \
  --save_weight ppo_smoke
# 建议额外日志: rewards.var(unbiased=False)
```

通过标准

- reward 均值和方差同时记录。
- actor/value/KL/clip 均为有限值。
- 样本检查未发现明显 reward hacking。
- 固定通用集与目标集均有对照。

高频误区与教师追问

误区 reward 上升就宣布能力提升。reward 可能只捕捉长度、格式或奖励模型偏好。

资源风险 PPO 同时驻留多个模型；按 SFT 单模型显存估算会严重低估。

形成性评价（含参考答案）

Q1. PPO 为什么需要 old policy logprob?

参考答案：用新旧概率比衡量本次更新相对 rollout 策略的变化，并进行 clip。

Q2. 为什么还需要 reference，而不是只用 PPO old policy?

参考答案：old policy 用于重要性比；reference 提供更长期的语言能力/行为锚点与 KL 约束。

课后任务

1. 构造三种 reward hacking 输出，并说明现有长度/thinking/重复规则会给出怎样的信号。
2. 画一个 PPO 失败诊断决策树，至少包含 reward 方差、KL、clip、value loss、长度和重复。

模块 12 GRPO 与 CISPO

建议学时	课程层级	先修模块	最低课堂产出
2	进阶	模块 11	group reward std 与 ratio 退化诊断

学习目标

- 解释同一 prompt 多回答的组内标准化 advantage。
- 比较 GRPO clip objective 与 CISPO 截断权重。
- 识别 num_generations=1、同组奖励相同与 rollout policy 滞后的退化。
- 补充原始 group reward std 日志并据此决定是否继续训练。

12.1 组内相对优势

每个 prompt 默认生成 6 个回答。GRPO 不训练 critic，而是用同组相对奖励提供基线；若同组 reward 没有差异，所有 advantage 接近 0，策略梯度消失。

$$A_i = (R_i - \text{mean}(R_{\text{group}})) / (\text{std}(R_{\text{group}}) + 10^{-4})$$

同一 completion 的所有有效 token 共用这个序列级 advantage。

12.2 GRPO 与 CISPO

方法	核心权重	当前默认/注意
GRPO	$\min(rA, \text{clip}(r)A)$	经典上下界 clip
CISPO	$\min(r, \varepsilon_{\text{high}}). \text{detach}() \cdot A \cdot \text{logp}$	脚本默认 loss_type=cispo

方法	核心权重	当前默认/注意
共同项	reference KL + token 长度归一	默认 beta=0.1

命名陷阱 train_grpo.py 默认跑 CISPO。若两种实验都沿用 save_weight=grpo，会覆盖 out、checkpoint 与 resume。

课堂流程

时间	教师活动	学生活动	验收证据
0—30 分	手算一组 reward 的 advantage	完成标准化	和/均值接近 0
30—60 分	对比 GRPO/CISPO loss	判断正负 advantage	权重方向正确
60—90 分	演示三种退化与命名覆盖	补日志并诊断	group std 非零

课堂实验：原始组内 reward 方差监控

实验目标：让“是否有可学信号”成为训练的硬门槛。

- 1. 显式指定 loss_type 与不同 save_weight，避免覆盖。
- 2. 在 grouped_rewards 上计算每组 std，再对 prompt 求均值。
- 3. 分别测试 num_generations=1、全同 reward、正常 reward。
- 4. 比较 Torch 与较慢同步 SGLang 时 ratio 分布。

```
[python]
group_reward_std = grouped_rewards.std(
    dim=1, unbiased=False
).mean()
print('group_reward_std=', float(group_reward_std))
```

通过标准

- GRPO/CISPO 使用独立 save_weight。
- 日志包含原始 group_reward_std，而非只看归一化 advantage std。
- num_generations=1 的退化能被实验复现。
- 同组无差异时停止长跑并修改数据/奖励。

高频误区与教师追问

误区 归一化后的 advantages_std 看起来正常，就认为原始奖励有区分度。两者不是同一指标。

吞吐瓶颈 默认每步 2 prompts×6 completions，奖励模型逐回答串行打分，常是主瓶颈。

形成性评价（含参考答案）

Q1. num_generations=1 为什么几乎无法学习？

参考答案：组内均值等于自身、std 为 0，标准化 advantage 约为 0。

Q2. 为什么 SGLang 权重同步间隔会影响 ratio？

参考答案：rollout 可能来自滞后的 policy，new/old logprob 差增大。

课后任务

1. 为 GRPO 日志增加 reward 分项：RM、长度、thinking、重复，并分析哪一项主导组内差异。

模块 13 多轮 Agentic RL 与工具环境

建议学时	课程层级	先修模块	最低课堂产出
2	进阶	模块 2、5、12	工具轨迹、policy mask 与 reward 审计

学习目标

- 解释多轮 tool call→observation→继续生成的轨迹结构。
- 区分模型生成 token 与环境 observation，并正确设置 policy mask。
- 审计工具 schema、执行器、GT、最大轮数和 reward shaping 的一致性。
- 识别 eval、模拟工具、子串 GT、左截断与公网部署风险。

13.1 从单轮 completion 到环境轨迹

Agent 默认每个 prompt 生成 4 条轨迹，最多 3 轮。模型产生 tool_call，执行器返回 observation，模板把 observation 重新编码进上下文，再继续生成。策略 loss 只覆盖模型生成 token；工具 observation 的 mask 必须是 0。

trajectory = prompt → action(tool_call) → observation → action ... → final answer → delayed reward

13.2 schema、执行器与奖励必须是一套系统

- 训练执行器只认识 6 个固定工具；README/eval/WebUI 还可能展示其他工具。
- 天气、时间、汇率、翻译、单位换算多为硬编码模拟，不代表实时能力。

- 工具分支检查 name、必填字段、调用数与 GT，并按最终答案命中 GT 给奖。
- GT 先做普通子串匹配，"1" 可能误命中 "10"，存在投机空间。
- max_total_len 只在完整 rollout 后左截断，可能删掉原问题与工具 schema。

生产红线 calculate_math 使用受限 eval 仍只适合受控教学；生产系统应使用白名单解析器、独立沙箱、超时、权限与审计。

13.3 成本与验收

默认 batch=2、generations=4、最多 3 轮，最坏每 step $2 \times 4 \times 3 = 24$ 次 generate，且内部多处串行。Agent 训练先验收轨迹和 mask，再讨论奖励与吞吐；否则模型可能在错误环境上“学得很好”。

课堂流程

时间	教师活动	学生活动	验收证据
0—25 分	演示一条多轮轨迹	标 action/observation	角色顺序正确
25—55 分	把 token 分成 policy mask 1/0	审计 observation	工具返回为 0
55—75 分	拆 reward 与 GT 漏洞	设计 hacking 样例	能说明修复
75—90 分	成本、安全与出口题	提交审计清单	含最大轮数/超时

课堂实验：端到端工具轨迹审计

实验目标：证明工具名、参数、返回、上下文与 mask 全部一致。

1. 选择 calculate_math 一条样本，检查 tools schema 与执行器必填字段。
2. 开启 debug，保存每轮 context、completion、tool result 与 reward。
3. 额外打印 full_response_masks/completion_mask，确认 observation 为 0。
4. 构造 GT 子串误命中、无限工具循环和超长轨迹，验证防护。

```
[python]
# 伪代码：对解码后的 token 区间做审计
for span in trajectory_spans:
    print(span.role, span.token_range, span.policy_mask)
# assistant/tool_call: 1; tool observation: 0
```

通过标准

- tool name/arguments 与 schema 和执行器一致。

- observation token 的 policy mask 全为 0。
- 最大工具轮数、超时与失败返回有定义。
- GT 匹配不是模糊子串投机。

高频误区与教师追问

误区 工具 schema 里出现一个函数，就以为训练执行器能执行它。schema 是声明，执行器才是能力。

截断风险 完整轨迹结束后左截断可能删除 system、工具定义和原问题；严格 token budget 应在每轮重建 prompt 时实施。

形成性评价（含参考答案）

Q1. 为什么 tool observation 不参与 policy loss?

参考答案：它由环境产生，不是策略选择的 token；把它训练成 action 会混淆 credit assignment。

Q2. 为什么调用数等于 GT 数不一定合理？

参考答案：一个工具调用可能返回多个答案，或多个调用共同解决一个 GT，数量不是普适语义。

课后任务

1. 为一个真实工具设计统一 schema、执行器接口、错误码、权限和审计字段。

模块 14 推理、API、转换与部署

建议学时	课程层级	先修模块	最低课堂产出
2	核心/进阶	模块 2—4、8	转换前后对照 + 本地 API 验收

学习目标

- 区分原生 .pth 与 Transformers 目录的加载参数边界。
- 解释 YaRN、生成上限、模型位置上限和可靠长上下文的区别。
- 完成模型格式转换并用 greedy token 对照定位第一处分叉。
- 识别轻量 OpenAI 风格 API、WebUI、Tool Calling 与第三方引擎的兼容/安全边界。

14.1 两条加载路线

当前 eval/API 通过路径中是否含小写 model 判断原生/Transformers，存在误判；同时无论设备都 model.half()，CPU/MPS 可能失败或极慢。这些是版本实现陷阱，不是模型原理。

路线	配置来源	LoRA/YaRN
原生 ./model + .pth	CLI 构造 MiniMindConfig	可加载原生 LoRA；CLI 可开 YaRN
Transformers 目录	目录 config.json	本地 LoRA 参数被忽略；YaRN 改 config

14.2 转换必须做语义等价验收

- convert_model.py 没有 argparse，底部默认将 full_sft_768.pth 转到 minimind-3。
- 避免覆盖已下载发布目录；输出到新目录。
- 同步核对 hidden/layers/use_moe/tokenizer/chat template。
- 用 do_sample=False 对比前 32 个新 token，定位第一处分叉。
- FP16 与不同 generate 实现可能不逐 bit 相同，但差异必须可解释。

14.3 API 与部署边界

本地服务只实现 POST /v1/chat/completions 的轻量协议：没有鉴权、限流、usage、模型列表、标准 [DONE] 等生产能力。非流式原生分支把 max_tokens 错传给 max_length，可能回落到 8192；流式 Tool Calling 可能同时发送原始标签和结构化 tool_calls。

目标	适用性	最低验证
SGLang/vLLM	CUDA 服务/rollout	版本、采样参数、tokenizer/template
llama.cpp/Ollama	GGUF/量化/本地	token ID、模板、中文、stop
MNN	端侧 4-bit HQQ	质量、速度、内存、绝对二进制路径
Streamlit WebUI	教学演示	模型扫描、HTML、eval、工具轮数

课堂流程

时间	教师活动	学生活动	验收证据
0—25 分	对比原生/Transformers 加载	判断参数是否生效	边界表
25—55 分	演示转换与 greedy 对照	定位 token 分叉	转换报告

时间	教师活动	学生活动	验收证据
55—75 分	启动 API 并测试 thinking/tool	检查响应字段	非流式通过
75—90 分	部署安全出口题	列公网前置条件	至少认证/限流/TLS

课堂实验：转换前后与 API 端到端验收

实验目标：证明转换没有破坏 tokenizer/template，并明确 API 兼容边界。

1. 把 .pth 转到全新的 Transformers 目录。
2. 用同一 prompt、tokenizer、greedy、32 tokens 比较原生/转换模型。
3. 启动 8998 本地服务，使用非流式请求测试文本、thinking、tool_calls。
4. 加入输入长度、最大输出、并发与未授权访问的失败测试。

```
[bash]
curl http://localhost:8998/v1/chat/completions \
  -H 'Content-Type: application/json' \
  -d '{"model": "minimind", "messages": [{"role": "user", "content": "解释 GQA"}], "stream": false, "max_tokens": 256}'
```

通过标准

- 转换目录与发布模型目录分离。
- 转换前后 tokenizer/template 一致。
- greedy 分叉位置已记录并可解释。
- API 不直接暴露公网；兼容差异有清单。

高频误区与教师追问

误区 能启动就等于转换成功。真正验收要覆盖 token、模板、输出和量化后任务质量。

安全风险 0.0.0.0、无鉴权、remote code、pickle、eval 与 unsafe HTML 组合不应直接进入公网。

形成性评价（含参考答案）

Q1. 为什么 --inference_rope_scaling 对 Transformers 目录可能不生效？

参考答案：Transformers 分支从目录 config 构造模型，不使用本地 CLI 的原生 config。

Q2. 为什么流式 Tool Calling 暂建议非流式？

参考答案：当前服务可能先发原始标签 content，结束又发结构化 tool_calls，客户端会重复回填。

课后任务

1. 写一份 API 生产化差距清单：鉴权、TLS、限流、超时、并发、日志脱敏、请求体上限、工具权限。

模块 15 评测、安全与综合项目

建议学时	课程层级	先修模块	最低课堂产出
2	核心	所有已选模块	固定回归集、模型卡式报告与项目答辩

学习目标

- 建立接线、协议、能力、系统四层评测。
- 在同模型家族内固定生成条件，跨 tokenizer 时避免直接比较 PPL。
- 为 RL 同时评估 reward、KL、长度、重复、目标成功、通用回归与人工盲评。
- 提交包含版本、许可、安全边界、失败样例和复现步骤的综合报告。

15.1 四层评测

同一 MiniMind checkpoint 比较必须固定 tokenizer/template、prompt/system、thinking、max tokens、temperature、top-p/top-k、seed/采样次数和截断。跨模型家族应各用原生 tokenizer/template，报告 token 数；跨 tokenizer 的语言建模效率优先比较 Bits Per Byte。

层级	核心问题	示例指标
接线层	能否正确运行与保存？	load/resume、NaN、shape
协议层	模板和结构是否正确？	mask、thinking、tool JSON
能力层	任务是否做对？	知识、推理、代码、目标域
系统层	能否稳定交付？	tokens/s、显存、延迟、并发、安全

15.2 最小回归集与 RL 评测

- 建议 50—200 条：模板/多轮 10%，中英文知识 15%，可验证数学 15%，代码 10%，目标域 20%，工具 20%，对抗/超长/非法 schema 10%。
- RL 需同时画 reward/方差、KL/clip、长度/重复、格式合法率、目标正确率、通用回归和人工盲评。

- 评测集污染与格式对齐会显著影响小模型分数；任何异常高分都要先查数据重叠。

15.3 综合项目设计

学生可选 Dense/MoE、SFT、LoRA、蒸馏、DPO 或一项 RL 分支，但必须只改变一个主要变量，提供基线、固定回归、失败分析、资源记录和安全/许可说明。项目不以模型分数高低为唯一目标，而以证据链质量评分。

最终答辩四问 你的输入契约是什么？你改变了哪个目标/参数？产物如何复现？哪类错误仍未解决？

课堂流程

时间	教师活动	学生活动	验收证据
0—25 分	用失败案例区分四层评测	分类指标	层级正确
25—55 分	共同设计回归集与判分器	给样本加标签	可判定且无训练泄漏
55—75 分	讲 RL/污染/安全	审计一份虚假高分报告	找出混杂变量
75—90 分	项目说明与出口答辩	提交一页实验设计	变量、基线、证据完整

课堂实验：构建最小回归与模型卡式报告

实验目标：把训练输出转化为可审查、可复现的结论。

1. 建立 50—200 条未参与训练的固定样本并标注类别/预期。
2. 固定解码条件，保存 baseline 与新 checkpoint 的原始输出。
3. 计算任务指标并按错误类型人工复核。
4. 写明数据/权重/依赖来源、已知风险、失败样例与复现命令。

```
[json]
# 建议每条评测记录字段
{
  'id': 'tool_001', 'category': 'tool',
  'prompt': '...', 'expected': {...},
  'generation_config': {...},
  'baseline_output': '...', 'candidate_output': '...',
  'score': 0, 'error_type': 'invalid_arguments'
}
```

通过标准

- 评测样本与训练数据去重/隔离。

- baseline 与 candidate 使用可比的生成条件。
- 原始输出、指标、错误类型均保留。
- 报告包含安全、许可证与部署边界。

高频误区与教师追问

误区 只汇报平均分，不保留原始输出与错误类型。小模型的格式错、重复和事实错需要分别处理。

污染风险 对齐数据与评测题重叠可制造异常高分；先做去重和来源审计，再解释能力。

形成性评价（含参考答案）

Q1. 为什么跨 tokenizer 不宜直接比较 PPL？

参考答案：PPL 以 token 为单位，分词粒度不同会改变尺度；可用 Bits Per Byte 等更可比指标。

Q2. RL 模型 reward 上升但正确率下降，如何结论？

参考答案：优化命中了奖励代理但未提升目标能力，属于 reward misalignment/hacking，应判失败并重置奖励/数据。

课后任务

1. 完成综合项目并按附录评分量表自评。

附录 A 知识点—源文件映射

本表帮助教师回到原教程查实现细节。教学模块按依赖重排，因此不与源文件章节一一对应。

教学模块	源文件主要章节	补充关注
1 全局地图/环境	§0—2、§23、§27—30	路径、许可、版本不一致
2 Tokenizer/协议	§3、§26.1	ID、模板、三类长度
3 模型 forward	§4.1—4.3、§26.2	GQA/RoPE/cache/SDPA
4 MoE/loss/generate	§4.4—4.7	aux、shift、解码边界
5 Dataset/mask	§5	五类契约与截断
6 训练工程	§6、§24	DDP、精度、保存恢复
7 Pretrain	§7、§25 里程碑 5	smoke 与成本口径
8 SFT	§8、§25 里程碑 6	thinking/tool 联合验收

教学模块	源文件主要章节	补充关注
9 LoRA/蒸馏	§9—10、§26.3	手写 LoRA 与教师门槛
10 DPO	§11	序列求和与长度偏差
11 PPO	§12—13、§26.4	reward、rollout、KL/value
12 GRPO/CISPO	§14	默认 CISPO 与组内 std
13 Agentic RL	§15、§17	执行器/schema/GT/mask
14 推理与部署	§16—21、§23	YaRN、API、转换、引擎
15 评测/项目	§22、§29	四层评测与完成清单

附录 B 核心公式速查

主题	公式	课堂解释
注意力	$\text{softmax}((QK^T/\sqrt{d})+M)V$	M 包含因果/必要的 padding mask
RMSNorm	$x/\sqrt{(\text{mean}(x^2)+\epsilon)}\cdot w$	实现常用 FP32 累积
Causal CE	$-\sum_t \log p(x_t x_{<t})$	模型内部 shift; -100 忽略
全局 batch	$\text{micro}\times\text{world_size}\times\text{accum}$	RL 另乘 rollout/更新结构
LoRA	$Wx+BAx$	当前 rank=16, B=0 初始化
蒸馏	$\alpha\text{CE}+(1-\alpha)T^2\text{KL}$	teacher 冻结, 同 tokenizer/vocab
DPO	$-\log\sigma(\beta[(\Delta\pi)-(\Delta\text{ref})])$	当前序列 logprob 求和
PPO	$\min(rA,\text{clip}(r)A)$	另含 value 与 reference KL
GRPO advantage	$(R_i-\text{mean_group})/(\text{std_group}+\epsilon)$	group std=0 时退化
CISPO	$\min(r,\epsilon_high).\text{detach}()\cdot A\cdot\text{logp}$	当前 train_grpo 默认

附录 C 阶段默认与产物速查（版本特定）

使用提醒 下表来自所分析源文件对应版本。教学时先打印实际 `argparse --help`/代码默认，再使用本表。

阶段	关键默认	前置权重	典型产物
----	------	------	------

阶段	关键默认	前置权重	典型产物
Pretrain	2 ep; b32; acc8; lr5e-4; len340	none	pretrain_768.pth
Full SFT	2 ep; b16; lr1e-5; len768	pretrain	full_sft_768.pth
LoRA	10 ep; b32; lr1e-4; len340; r16	full_sft	lora_<name>_768.pth
Distill	6 ep; b32; lr5e-6; α 0.5; T1.5	student Dense / teacher MoE	full_dist_768.pth
DPO	1 ep; b4; lr4e-8; β 0.15; len1024	full_sft	dpo_768.pth
PPO	b2; actor3e-7; gen1024; KL0.02	full_sft + RM	ppo_actor_768.pth
GRPO/CISPO	b2; gen6; β 0.1; default CISPO	full_sft + RM	显式分开命名
Agent	b2; gen4; 3 turns; total2500	full_sft (+RM)	agent_<loss>_768.pth

附录 D 排障决策表

现象	优先检查	不要先做什么
SFT loss NaN	active labels 是否全 -100；模板边界与截断	不要先盲目调学习率
权重找不到	工作目录、命名、save_dir 与固定 ../out 加载	不要复制多个不明版本覆盖
MoE 不收敛	CE+aux、每专家 token count、DDP unused	不要只看总 loss
GRPO 不学习	原始 group reward std、num_generations、奖励分辨率	不要仅延长 epoch
RL reward 上升但质量差	长度/重复/KL/目标准确/通用回归	不要宣布能力提升
API 输出过长	非流式 max_tokens→max_new_tokens 接线	不要误以为客户端没传
Tool Call 重复	流式原始标签+结构化 tool_calls 双发	修复前不要用流式回填
CPU/MPS 推理异常	强制 model.half() 与算子支持	不要把 CUDA 结论外推
长上下文崩溃	prompt+generation、RoPE buffer、实际截断	不要只看 tokenizer 131072
转换后输出漂移	config/tokenizer/template/dtype/第一处分叉	不要只看目录能加载

附录 E 综合项目评分量表

维度	权重	优秀证据	主要扣分点
可复现性	20	commit/环境/命令/seed/产物齐全	只给截图或口头描述
概念解释	20	能把张量、目标与实现对应	堆术语、无法解释 shape/loss
数据与协议	15	mask/template/schema 有审计	没有 active label/坏样本检查
实验设计	15	单一主变量、基线与预算公平	同时改多项且无消融
评测与错误分析	20	固定回归、原始输出、分类型指标	挑选漂亮样例或只报平均分
安全与许可	10	来源、许可、风险、部署边界清楚	公网暴露、eval/remote code 无说明

提交物清单

- 代码提交与环境锁定信息；
- 数据卡：来源、许可、字段、清洗、训练/评测隔离；
- 模型与训练配置、权重命名、日志和资源记录；
- 至少一个 baseline 与一个 candidate；
- 固定回归集、原始输出、指标、错误类型与失败样例；
- 安全边界、许可证说明、尚未解决的问题和复现命令。

附录 F 十道概念出口题

1. 131072、32768、max_seq_len 分别是什么？

答案：tokenizer 元数据、默认 RoPE 缓冲长度、本次训练/截断长度。

2. Tool/Think 为什么 decode 后仍保留？

答案：它们是 added token 但未标 special，便于参与 loss 与文本解析。

3. GQA 减少什么？

答案：减少独立 K/V heads 与 cache，不减少 Q heads。

4. 198M-A64M 如何解释？

答案：约 198M 总存储参数，每 token 只激活约 64M 路径。

5. 模型和 Dataset 谁负责 next-token shift？

答案：Pretrain/SFT 由模型内部 shift；DPO 数据因不走 CE 自行 shift。

6. SFT 全 -100 为什么危险？

答案：没有有效目标，mean CE 可能 NaN，且样本不提供梯度。

7. DPO 为什么没有 critic？

答案：直接用 policy/reference 的偏好 margin 构造可微目标。

8. GRPO 最关键的先验指标是什么？

答案：原始组内 reward std；接近 0 表示无相对学习信号。

9. 为什么 reward 上升不等于能力上升？

答案：reward 是代理，可被长度、格式、重复或 RM 偏好投机。

10. 部署前最小安全边界？

答案：认证、TLS、限流、超时、输入/工具轮数上限、日志脱敏、可信权重/remote code、沙箱。

附录 G 来源、版本与使用边界

本讲义依据用户提供的 GitHub 文件进行知识分析与教学重构，不替代仓库代码、上游文档或各依赖的官方说明。源文件本身标明其逐文件整理基于 upstream commit 89d674b8；本次分析读取的 fork 文件位于 commit 2e2ed4cc。版本更新后，应重新核对默认参数、路径和已知问题。

源文件：[MiniMind-完整学习实践教程.md](#)

上游仓库：[jingyaogong/minimind](#)

许可说明 仓库 Work 使用 Apache-2.0；数据、奖励模型、依赖、模型权重和字体各有自己的许可。教学或再分发前应分别核对。此处不是法律意见。